

Lab 07 – Singly Linked List

Task 01:

You are assigned the task of creating a news feed system for a social media platform using a singly linked list in C++. Each post should be represented as a node containing both the post content and a timestamp.

Your program should support the following operations:

1. *Adding a new post to the end of the feed*
2. *Removing a post by its content*
3. *Displaying all posts in chronological order*
4. *Total Count of Posts*
5. *Search a post by its content*

In your `main()` function, please demonstrate the functionality by:

- *Adding seven posts on any topic of your choice*
- *Removing two posts as requested by the user*
- *Displaying all remaining posts to verify that the system operates correctly.*
- *Display the total no. of posts.*
- *Search any one post as requested by user*

Code:

```
/*News Feed*/
```

```
#include<iostream>
```

```
using namespace std;
```

```
class Node{
```

```
public:
```

```
    string content;
```

```
    int timeStamp;
```

```
    Node* next;
```

```
    Node(string content, int timeStamp){
```

```
        this->content = content;
```

```
        this->timeStamp = timeStamp;
```

```
        next = nullptr;
```

```
    }
```

```
};
```

```
class NewsFeed{
```

```
private:
```

```
    Node* head;
```

```
public:
```

```
NewsFeed(){ head = nullptr; }

/**
 * 1. Adding a new post to the end of the feed
 */
void addPost(string content, int timeStamp){
    if(head == nullptr){
        head = new Node(content, timeStamp);
        return;
    }
    Node* newNode = new Node(content, timeStamp);
    Node* mover = head;

    while(mover->next != nullptr) mover = mover->next;

    mover->next = newNode;
}

/**
 * 2. Removing a post by its content
 */
void removePost(string content){
    if(head == nullptr){
        cout << "There are no news in the list.\n";
        return;
    }

    if(head->next == nullptr){

        if(head->content == content){
            Node* temp = head;
            head = nullptr;
            cout << "Content: '" << temp->content << "' Timestamp: '" <<
temp->timeStamp << "', deleted successfully!\n";
            delete temp;
            return;
        }
        else{
            cout << "There is no news wiht this content in the list.\n";
            return;
        }
    }

    if(head->content == content){
        Node* temp = head;
```

```

        head = head->next;
        cout << "Content: '" << temp->content << "' Timestamp: '" << temp-
>timeStamp << "', deleted successfully!\n";
        delete temp;
        return;
    }

    Node* pre = head;
    while(pre->next != nullptr){
        if(pre->next->content == content) break;
        pre = pre->next;
    }

    // we reached at last node
    if(pre->next == nullptr){

        // the content is also which we want to delete..
        if(pre->content == content){
            cout << "Content: '" << pre->content << "' Timestamp: '" << pre-
>timeStamp << "', deleted successfully!\n";
            delete pre;
            return;
            // there is a BUG
        }
        else{
            cout << "There is no news with this content in the list.\n";
            return;
        }
    }

    Node* toBeDeleted = pre->next;
    pre->next = toBeDeleted->next;
    cout << "Content: '" << toBeDeleted->content << "' Timestamp: '" <<
toBeDeleted->timeStamp << "', deleted successfully!\n";
    delete toBeDeleted;
}

/**
 * 3. Displaying all posts in chronological order
 */
void displayAllPosts(){
    if(head == nullptr){
        cout << "There are no news in the list to display.\n";
        return;
    }
}

```

```

    Node* mover = head;
    cout << "\tHeadlines by Hasan Newz.\n";
    int sno = 1;
    while(mover){
        cout <<sno++<< ". Content: '" << mover->content << "' Timestamp: '"
<< mover->timeStamp << "'.\n";
        mover = mover->next;
    }
}

/**
 * 4. Total Count of Posts
 */
int totalPost(){
    if(head == nullptr) return 0;

    int count = 0;
    Node* mover = head;
    while(mover){
        count++;
        mover = mover->next;
    }
    return count;
}

/**
 * 5. Search a post by its content
 * @return address of that node
 */
Node* search(string content){
    if(head == nullptr) return nullptr;

    Node* mover = head;
    while(mover != nullptr){
        if(mover->content == content) break;
        mover = mover->next;
    }

    if(mover == nullptr) return nullptr;

    return mover;
}

};

int main(){

```

```
NewsFeed feed = NewsFeed();

// — 1. Add seven posts
cout << "=====\n";
cout << "          ADDING POSTS TO FEED\n";
cout << "=====\n";

feed.addPost("Post 1", 1);
feed.addPost("Post 2", 2);
feed.addPost("Post 3", 3);
feed.addPost("Post 4", 4);
feed.addPost("Post 5", 5);
feed.addPost("Post 6", 6);
feed.addPost("Post 7", 7);

cout << "\n7 posts added successfully!\n";

// — 2. Display all posts
cout << "\n=====\n";
cout << "          ALL POSTS (CHRONOLOGICAL)\n";
cout << "=====\n";
feed.displayAllPosts();

// — 3. Total count
cout << "\n=====\n";
cout << "          TOTAL POST COUNT\n";
cout << "=====\n";
cout << "Total posts: " << feed.totalPost() << "\n";

// — 4. Remove two posts as requested by user
cout << "\n=====\n";
cout << "          REMOVING POSTS\n";
cout << "=====\n";

string removeTarget;

cout << "Enter content of post to remove (1st): ";
getline(cin, removeTarget);
feed.removePost(removeTarget);

cout << "Enter content of post to remove (2nd): ";
getline(cin, removeTarget);
feed.removePost(removeTarget);
```

```
// — 5. Display remaining posts
cout << "\n===== \n";
cout << "          REMAINING POSTS\n";
cout << "===== \n";
feed.displayAllPosts();

// — 6. Updated count
cout << "\n===== \n";
cout << "          UPDATED TOTAL POST COUNT\n";
cout << "===== \n";
cout << "Total posts: " << feed.totalPost() << "\n";

// — 7. Search a post as requested by user
cout << "\n===== \n";
cout << "          SEARCH A POST\n";
cout << "===== \n";

string searchTarget;
cout << "Enter content to search: ";
getline(cin, searchTarget);
if(feed.search(searchTarget) != nullptr) cout << "found.";

cout << "\n===== \n";
cout << "          DONE!\n";
cout << "===== \n";

}
```

Output:

```
=====
      ADDING POSTS TO FEED
=====

7 posts added successfully!

=====
      ALL POSTS (CHRONOLOGICAL)
=====
      Headlines by Hasan Newz.
1. Content: 'Post 1' Timestamp: '1'.
2. Content: 'Post 2' Timestamp: '2'.
3. Content: 'Post 3' Timestamp: '3'.
4. Content: 'Post 4' Timestamp: '4'.
5. Content: 'Post 5' Timestamp: '5'.
6. Content: 'Post 6' Timestamp: '6'.
7. Content: 'Post 7' Timestamp: '7'.

=====
Total posts: 7

=====
      REMOVING POSTS
=====
Enter content of post to remove (1st): Post 1
Content: 'Post 1' Timestamp: '1', deleted successfully!
Enter content of post to remove (2nd): Post 2
Content: 'Post 2' Timestamp: '2', deleted successfully!
```

```
=====
REMAINING POSTS
=====
Headlines by Hasan Newz.
1. Content: 'Post 3' Timestamp: '3'.
2. Content: 'Post 4' Timestamp: '4'.
3. Content: 'Post 5' Timestamp: '5'.
4. Content: 'Post 6' Timestamp: '6'.
5. Content: 'Post 7' Timestamp: '7'.

=====
UPDATED TOTAL POST COUNT
=====
Total posts: 5

=====
SEARCH A POST
=====
Enter content to search: Post 3
found.

=====
DONE!
=====
```


Task 02:

The Recent Books Read system is a program that keeps track of the books the user has read recently. This makes it easier for users to quickly locate and review the books they've read without having to search through various genres. Create a program in C++ that implements the above scenario using a singly linked list. The following functions are required:

- a. A new recent book is added with the title, genre, and the time it was read. (Add any 5 books)
- b. Display all recent books that are stored in the list.
- c. Delete a recent book & display the remaining books in the list

Code:

```
/*Reading Activity Log*/

#include<iostream>
using namespace std;

struct Book{
    string title;
    string genre;
    int time;
};

class Node{
public:
    Book data;
    Node* next;

    Node(Book data){
        this->data = data;
        next = nullptr;
    }
};

class ReadingActivityLog{
private:
    Node* head;

public:
    ReadingActivityLog(){ head = nullptr; }

    void addBook(Book data){
        if(head == nullptr){
            head = new Node(data);
        }
    }
};
```

```
        return;
    }

    Node* newNode = new Node(data);
    newNode->next = head;
    head = newNode;
}

void displayRecentBooks(){
    if(head == nullptr){
        cout << "List is empty.\n";
        return;
    }

    Node* temp = head;
    while(temp){
        cout << "Title: " << temp->data.title << " | Genre: " << temp->data.genre << " | Time: " << temp->data.time << "\n";
        temp = temp->next;
    }
}

void deleteBook(){
    if(head == nullptr){
        cout << "List is empty.\n";
        return;
    }

    Node* temp = head;
    head = temp->next;
    delete temp;
}

};

int main(){
    ReadingActivityLog obj;

    obj.addBook({"first book", "reality", 1234});
    obj.addBook({"second book", "horror", 1234});
    obj.addBook({"third book", "fiction", 1234});
    obj.addBook({"fourth book", "fantasy", 1234});
    obj.addBook({"fifth book", "thriller", 1234});

    cout << "\n\tAll books in the list\n";
```

```
    obj.displayRecentBooks();

    obj.deleteBook();
    cout << "\n\tRemaining books in the list\n";
    obj.displayRecentBooks();

}
```

Output:

```
    All books in the list
Title: fifth book | Genre: thriller | Time: 1234
Title: fourth book | Genre: fantasy | Time: 1234
Title: third book | Genre: fiction | Time: 1234
Title: second book | Genre: horror | Time: 1234
Title: first book | Genre: reality | Time: 1234

    Remaining books in the list
Title: fourth book | Genre: fantasy | Time: 1234
Title: third book | Genre: fiction | Time: 1234
Title: second book | Genre: horror | Time: 1234
Title: first book | Genre: reality | Time: 1234
```

Task 03:

BeanBuzz is a popular coffee shop located in the heart of the city. With a constant stream of customers, managing orders efficiently is essential to keep the business running smoothly. To improve customer service, the management has decided to implement a digital queue system.

Your task is to develop a C++ program that utilizes a singly linked list to manage the queue of customer orders at BeanBuzz. The program should include the following functionalities:

- a. Placing Order
- b. Serving Order
- c. Display Queue Status
- d. Display Order Details
- e. Exit()

Code:

```
/*BeanBuzz Queue Management System*/
```

```
#include<iostream>
#include<string>
using namespace std;

struct Order{
    int no;
    string customerName;
    string drinkType;
    string customization;
};

class Node{
public:
    Order data;
    Node* next;

    Node(Order data){
        this->data = data;
        next = nullptr;
    }
};

class BeanBuzzQueue{
private:
    Node* head;
    int f, r;

public:
```

```
BeanBuzzQueue(){
    head = nullptr;
    f = r = -1;
}

void placeOrder(Order od){
    if(f == -1){
        f = r = 0;
        head = new Node(od);
        cout << "Order placed successfully. Order number: " << od.no << "\n";
    }

    // insertion at the end case
    else{
        r++;
        Node* newNode = new Node(od);

        Node* temp = head;
        while(temp->next != nullptr) temp = temp->next;
        temp->next = newNode;

        cout << "Order placed successfully. Order number: " << od.no << "\n";
    }
}

void serveOrder(){
    if(f == -1) cout << "There are no orders in the list.\n";
    else{
        Node* temp = head;
        head = head->next;

        cout << "Order served successfully. Served order number " << temp-
>data.no << "\n";
        delete temp;
        f++;
    }

    if(head == nullptr) // queue is now empty, reset
        f = r = -1;
}

void displayQueueStatus(){
    if(f == -1) cout << "There are no orders in the queue.\n";
    else{
        cout << "\t---Orders in the Queue---\n\n";
    }
}
```

```

        Node* mover = head;
        while(mover){
            cout << "Order No. " << mover->data.no << " Customer name: " <<
mover->data.customerName << " | Drink type: " <<
            mover->data.drinkType << " | Customization: " << mover-
>data.customization << "\n";

            mover = mover->next;
        }
    }
}

```

```

Node* searchOrder(int target){
    if(f == -1) return nullptr;

    Node* mover = head;
    while(mover){
        if(mover->data.no == target) return mover;
        mover = mover->next;
    }

    return nullptr;
}
};

```

```

int main(){
    int choice = 0;
    int totalOrders = 0;
    BeanBuzzQueue q;
    cout << "\n\t***Welcome to BeanBuzz Queue Managment System***\n\n";

    do{
        cout << "\n\t---MENU---\n";
        cout << "1. Place Order.\n";
        cout << "2. Serve Order.\n";
        cout << "3. Display Queue Status.\n";
        cout << "4. Display Order Details\n";
        cout << "5. Exit\n";
        cout << "Enter choice: ";
        cin >> choice;
        cin.ignore();
        cout << "\n";

        switch (choice){

```

```
case 1:{
    Order newOrder;
    cout << "Enter customer name: ";
    getline(cin, newOrder.customerName);
    cout << "Enter drink type: ";
    getline(cin, newOrder.drinkType);
    cout << "Enter customization: ";
    getline(cin, newOrder.customization);
    newOrder.no = ++totalOrders;

    q.placeOrder(newOrder);
}
break;

case 2:
    q.serveOrder();
break;

case 3:
    q.displayQueueStatus();
break;

case 4:{
    int target = 0;
    cout << "Enter order no. to search: ";
    cin >> target;
    cin.ignore();

    Node* order = q.searchOrder(target);

    if(!order) cout << "There is no order with this order number.\n";
    else{
        cout << "\tOrder details\n";
        cout << "No. " << order->data.no << "Customer name: " <<
order->data.customerName << " | Drink type: " <<
        order->data.drinkType << " | Customization: " << order-
>data.customization << "\n";
    }
}
break;

case 5:
    cout << "Closing program.\n";
break;
```

```
        default:
            cout << "Please enter a valid choice.\n";
            break;
    }

    }while(choice != 5);
}
```

Output:

```
***Welcome to BeanBuzz Queue Managment System***

    ---MENU---
1. Place Order.
2. Serve Order.
3. Display Queue Status.
4. Display Order Details
5. Exit
Enter choice: 1

Enter customer name: hasan
Enter drink type: water
Enter customization: none.
Order placed successfully. Order number: 1

    ---MENU---
1. Place Order.
2. Serve Order.
3. Display Queue Status.
4. Display Order Details
5. Exit
    ---Orders in the Queue---

Order No. 1 Customer name: hasan | Drink type: water | Customization: none.

    ---MENU---
```



```
    ---MENU---
1. Place Order.
2. Serve Order.
3. Display Queue Status.
4. Display Order Details
5. Exit
Enter choice: 2

Order served successfully. Served order number 1

    ---MENU---
1. Place Order.
2. Serve Order.
3. Display Queue Status.
4. Display Order Details
5. Exit
Enter choice: 4

Enter order no. to search: 1
There is no order with this order number.

    ---MENU---
1. Place Order.
2. Serve Order.
3. Display Queue Status.
4. Display Order Details
5. Exit
Enter choice: 5

Closing program.
```

Task 04:

Imagine a city's transportation system with a circular train route. The train stations are arranged in a circular route, meaning that the first station is connected to the last station to form a loop. The train starts from a station, travels to the next station, and continues in a loop without having to stop at the "end." Design a Circular Train Route System using a Circular Linked List. The system should support the following operations:

1. *Add a new station to the circular route.*
2. *Display the entire route of stations.*

Code:

```
/*Train Route Manager*/

#include<iostream>
using namespace std;

class Node{
public:
    string station;
    Node* next;

    Node(string station){
        this->station = station;
        next = nullptr;
    }
};

class TrainRoute{
private:
    Node* head;
    Node* tail;

public:
    TrainRoute(){ head = tail = nullptr; }

    /**
     * this fn is considering indexing from 1
     */
    void insertAtPosition(int pos, string name){
        if(pos < 1){
            cout << "Invalid Position.\n";
            return;
        }
    }
};
```

```

    if(head == nullptr && pos != 1){
        cout << "Position out of bounds.\n";
        return;
    }

    /**
     * handling pos == 1 here for both conditions
     * 1. linked list is empty
     * 2. any other situation of ll
     *
     * the loop we are writing willn't work if pos == 1;
     * beacise our loop will never run and first node will be considered
previous
    */
    if(pos == 1){
        if(head == nullptr){
            head = tail = new Node(name);
            tail->next = head;
        } else{
            Node* newNode = new Node(name);
            newNode->next = head;
            head = newNode;
            tail->next = head;
        }
        return;
    }

    Node* newNode = new Node(name);
    Node* pre = head;

    for(int i = 1; i < pos-1; i++){
        pre = pre->next; // moving pre* first, beacise in first itr it is obv
on head...

        // if you are thinking that for pos == 2 so pre* will obviously be on
head...

        // no problem in that our loop willn't even run one time if pos ==
2...

        // this is in case we reach again to head...
        if(pre == head){
            cout << "Position out of bounds.\n";
            return;
        }
    }

```

```
    }
    newNode->next = pre->next;
    pre->next = newNode;

    if(pre == tail) tail = newNode;
}
/**
 * indexing from 1
 */
void deleteAtPosition(int pos){
    if(pos < 1){
        cout << "Invalid position.\n";
        return;
    }

    if(head == nullptr){
        cout << "List is empty.\n";
        return;
    }

    // now it is confirmed that our list exists, let's tackle pos == 1
    first...

    if(pos == 1){
        if(head->next == head){
            delete head;
            head = tail = nullptr;
        } else{
            Node* temp = head;
            head = head->next;
            tail->next = head;
            delete temp;
        }
        return;
    }

    Node* pre = head;

    for(int i = 1; i < pos-1; i++){
        pre = pre->next;

        if(pre == head){ // we traversed completely on list and now have come
            back to head again...
            cout << "Position out of bounds.\n";
            return;
        }
    }
}
```

```
    }
}

if(pre == tail){ // this can also be handled in loop
    cout << "Position out of bounds.\n";
    return;
}

if(pre->next == tail){ // we reached to second last node
    delete tail;
    tail = pre;
    tail->next = head;
} else{
    Node* toBeDeleted = pre->next;
    pre->next = toBeDeleted->next;
    delete toBeDeleted;
}
}

void displayList(){
    if(head == nullptr){
        cout << "List is empty.\n";
        return;
    }

    Node* temp = head;
    while(true){
        cout << temp->station << " ";
        temp = temp->next;

        if(temp == head) break;
    }
    cout << "\n";
}

};

int main(){

}
```